



Make comparisons using operators

You've learned about data types, like integer, string, and float. Another data type is Boolean data. This is data that has only two possible values, usually true or false. The word "Boolean" comes from George Boole, a 19th-century English mathematician. Every time you compare things in Python, the result is Boolean type data. Data professionals use Boolean data every day to control logical flows in their code. In previous lessons, you discovered how Python can be used like a calculator for basic arithmetic.

Now we'll find out how to use the power of Python to compare values with comparators and operators. Comparators are operators that compare two values and produce Boolean values. All right, now let's consider an example. If we print 10 is greater than 1, a comparator produces the result, a Boolean value, True. There are six comparators in Python. They let us confirm whether something is: greater than, greater than or equal to, less than, less than or equal to, equal to, or not equal to something else. Data professionals take the results of comparator expressions and use them to make decisions about data assigned to these descriptions.

Here's an example. Cat is not, in fact, equal to dog. So it produces the Boolean value False. Now, let's pair an exclamation mark and an equal sign, which is the not equals comparator. So, the comparator checks that 1 isn't equal to 2 and produces the Boolean

value of True. As we've learned, the plus operator doesn't work between integers and strings. So, let's consider what will happen if we try to compare an integer and a string. Yep, another `TypeError`. The good news is that Python also has a set of logical operators. Logical operators are operators that connect multiple statements together and perform more complex comparisons. Examples of these are the words `and`, `or`, and `not`. These operators allow you to connect multiple statements together and perform more complex comparisons. The `and` operator needs both expressions to be true to return a True result. Here we're comparing strings.

When used on strings of text, comparators evaluate the first letter of each string, with A being least and Z being greatest. If two strings have the same first letter, the second letter will be compared. In this case, the Y in "yellow" is greater than the C in "cyan," but the B in "brown" doesn't come after the M in "magenta." So this means that the first statement is true, but the second one isn't true. So, if only part of an expression is true, the result of the whole `and` statement is false. Or statements are the opposite. If we use the `or` operator, the expression will be True if either of the expressions are true, and False only when both expressions are false.

Try it out. Print this code: Open parenthesis, 25, the greater than comparator, 50, the `or` operator, 1, the not equal comparator, 2, close parenthesis. 25 is definitely not greater than 50, but 1 is not equal to 2. So, in the end, the whole expression is true. Now, the `not` operator inverts the value of the expression that follows it. If it's true, it becomes false. If it's false, it becomes true. Because there is a `not` statement in front of 42 equals the string "Answer," the result is true.

Comparators and logical operators are very useful in the data field because they make it possible to write much more complex code. Later, I'll be back to demonstrate some examples of these expressions. Keep practicing and reviewing comparators and operators. I'll be with you again soon.

Boolean

A data type that has only two possible values, usually true or false

Comparators

Operators that compare two values and produce boolean values (True/False)

Python Comparators	Symbols
Greater than	>
Greater than or equal to	>=
Less than	<
Less than or equal to	<=
Equal to	==
Not equal to	!=

Logical operators

Operators that connect multiple statements together and perform more complex comparisons

Logical operators

- **and**
- **or**
- **not**

The **and operator** needs both expressions to be true to return a True result.

If we use the **or operator** the expression will be True if either of the expressions is true, and False only when both expressions are false.

The **not operator** inverts the value of the expression that follows it.

Question



Which of the following words are examples of logical operators? Select all that apply.

☐ greater than

☒ and

✔ **Correct**

Logical operators include the words *and*, *or*, and *not*. Logical operators allow you to connect multiple statements together and perform more complex comparisons.

☒ or

✔ **Correct**

Logical operators include the words *and*, *or*, and *not*. Logical operators allow you to connect multiple statements together and perform more complex comparisons.

☒ not

✔ **Correct**

Logical operators include the words *and*, *or*, and *not*. Logical operators allow you to connect multiple statements together and perform more complex comparisons.

Continue

Skip